

スクリプトプログラミング演習 2

最終レポート

GitHub・ngrok・Jenkins 連携による
天気データ自動出力

学籍番号 HW25A066

氏名 嶋田 一步

提出日 2026 年 7 月 1 日

テーマ：天気データの出力 + JavaScript による拡張

1. 課題の目的

GitHub への push を契機として Jenkins を自動実行し、天気データを取得・集計して成果物を生成する仕組みを作成した。外部からローカル Jenkins へ Webhook を届けるために ngrok を利用する。

- 必須要素：Jenkins から天気データ出力処理を実行
- GitHub/ngrok 対応：push イベントを Webhook で Jenkins へ転送
- 加要素：JavaScript による HTML 生成、Discord 通知、単体テスト、成果物検証

2. システム構成

処理の流れ

GitHubのpushから成果物保存までを自動化

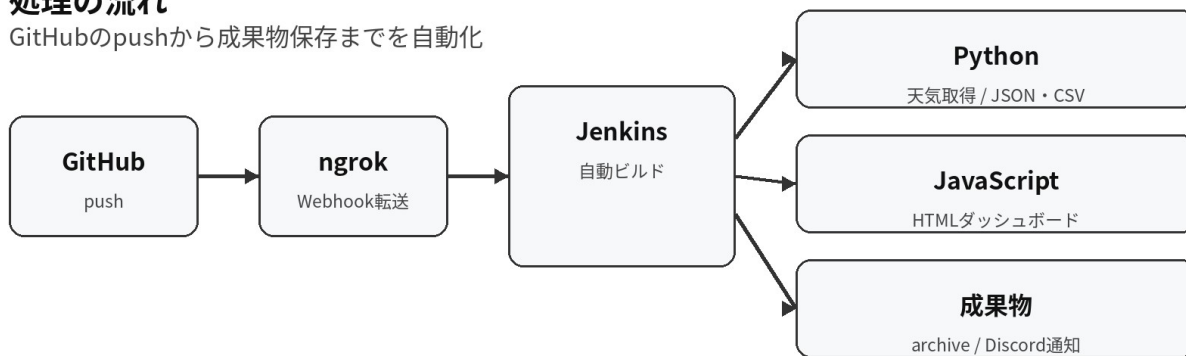


図1 GitHub から成果物保存までの処理

GitHub の Webhook URL には、ngrok で発行した URL の末尾へ「/github-webhook/」を追加する。Jenkins 側では GitHub hook trigger を有効にし、Jenkinsfile を Pipeline from SCM として読み込む。

3. 実装内容

ファイル	役割	主な処理
fetch_weather.py	Python	Open-Meteo 取得、障害時 fixture 切替、JSON/CSV 出力
src/weather_pipeline.py	Python	API URL 生成、日別集計、ファイル出力
generate_dashboard.js	JavaScript	JSON を読み込み、静的 HTML ダッシュボードを生成
verify_outputs.py	Python	JSON/CSV/HTML の存在・件数・内容を検証
notify_discord.py	Python	成功・失敗を Discord へ通知（設定時のみ）
Jenkinsfile	Pipeline	テスト、生成、検証、成果物保存を順番に実行

Jenkins の主なステージ

```

Checkout
Tool versions
Unit tests
Weather data output
JavaScript dashboard extension
Verify outputs
Archive artifacts

```

API へ接続できない場合でも、サンプルデータへ自動的に切り替えられるようにした。授業内デモでネットワーク障害が起きても、パイプライン全体の動作確認を継続できる。

4. 出力結果

出力は JSON・CSV・HTML・検証結果 JSON の 4 種類とした。HTML は授業で扱っていない JavaScript を使って生成している。

成果物	内容	用途
weather_data.json	地点、生成日時、7 日分の日別集計	プログラム間のデータ連携
weather_data.csv	日付・気温・湿度・降水量など	表計算ソフトで確認
index.html	カード形式と表形式の天気表示	ブラウザで結果確認
build_summary.json	検証結果とファイルサイズ	Jenkins の成功判定補助



図2 JavaScript で生成した HTML 出力（上部）

5. 結果

提出前のソース検証として、オフラインデータを使用してパイプラインを実行した。Python 単体テスト 5 件、データ生成、JavaScript 出力、成果物検証がすべて成功した。

Local verification - HW25A066

```
+ /opt/pyvenv/bin/python -m unittest discover -s tests -v
+ /opt/pyvenv/bin/python fetch_weather.py --offline
[weather] offline mode: data/sample_open_meteo.json
[weather] generated 7 days
[weather] JSON: output/weather_data.json
[weather] CSV : output/weather_data.csv
+ node tools/generate_dashboard.js
[dashboard] generated: output/index.html
[dashboard] rows: 7
+ /opt/pyvenv/bin/python tools/verify_outputs.py
[verify] success: 7 rows, 3 primary artifacts

Local pipeline completed successfully.
```

図 3 ローカル検証ログ

確認項目	結果
Python 単体テスト	5 件すべて OK
日別データ件数	7 日分
JSON・CSV 整合性	一致
HTML 必須文字列	確認済み
主成果物	3 ファイル + build_summary

6. GitHub • ngrok • Jenkins 連携

- Jenkins ジョブは「Pipeline script from SCM」で作成し、リポジトリ直下の Jenkinsfile を指定する。
- ngrok は Jenkins の 8080 番ポートを公開する。固定ドメインを使用すると Webhook URL を変更せずに済む。
- GitHub Webhook は Content type を application/json、push イベントのみ、Active を有効にする。
- push 後、ngrok に POST /github-webhook/ 200 OK が表示され、Jenkins が自動開始することを動画で示す。

Webhook URL 例

```
https://<自分の ngrok ドメイン>/github-webhook/
```

ngrok 起動例

```
ngrok http 8080  
ngrok http --url=<固定ドメイン>.ngrok-free.app 8080
```

7. 提出物

- プロジェクト一式（Jenkinsfile、Python、JavaScript、テスト、サンプルデータ）
- Jenkins 実行動画（GitHub、ngrok、Jenkins の連動が分かるもの）
- 出力4 ファイル（JSON、CSV、HTML、build_summary）
- 成功ビルドの Jenkins コンソールログ
- 本レポート

まとめ

本課題では、ソース管理、Webhook、CI、データ取得、テスト、成果物生成を一つの流れとして実装した。単にスクリプトを実行するだけでなく、失敗時の代替データ、成果物の自動検証、任意の Discord 通知を追加し、授業内容を拡張した。